Ders 7 Çalışma Özeti

Ders 7 Çalışma Özeti

Genel Konular

- Asıl ders yaklaşık 00:06'da başlar. Kaydın başı bekleme kısmıdır.
- Proje export ve import işlemleri
  - NetBeans'te projeyi `File > Export Project > To ZIP` ile dışa aktarın. Dosya adına `.zip` uzantısını yazın.
  - Kaynak kod `src` altındaki `.java` dosyalarıdır. `build` klasöründeki `.class` dosyaları bytecode'dur.
- Kopya yapıcı (copy constructor)
  - `Kitap` sınıfına `Kitap(Kitap k)` yapıcısı eklendi. Bu yapıcı, var olan bir kitabın kopyasını oluşturur.
  - İlkel alanlar ve referans alanlar kopyalanırken farklı davranır (sığ kopya).
  - C++'taki kopya yapıcı ve yıkıcı (destructor) kısaca gösterildi.
- Senaryodan sınıf ve özellik çıkarma
  - Bir üniversite bilgi sistemi senaryosundan öğrencilerle birlikte sınıflar çıkarıldı: üniversite, fakülte, bölüm, öğretim üyesi, yetki, derslik, malzeme, öğrenci, katalog ders, açılan ders.
- Adres defteri örneği (gereksinim, use case, sequence diagram, class diagram)
- Nesneler arası ilişkiler (relations between objects)
  - İlişki türleri: association, dependency, aggregation, composition, inheritance.
  - Bu derste association ve dependency işlendi. Tek yönlü (one-way) association kodlandı. Çift yönlü (two-way) association'a giriş yapıldı.

Hocanın Özellikle Vurguladığı Kısımlar

- IDE'nin kod üretme özelliklerini kullanın.
  - NetBeans'te `Insert Code` ile constructor, getter ve setter hızlıca oluşturulur.
- Referans tipli alanın kopyalanması, nesnenin kopyalanması demek değildir.
  - Kopya yapıcıda `yazar` referansı kopyalanır. İki kitap aynı `Yazar` nesnesini gösterir.
- UML'den kod yazarken ilişkileri koda çevirin.
  - Diyagramda `Customer` ile `Order` arasında "1 - *" ilişki görürseniz, `Customer` sınıfına `private` bir `Order` dizisi veya listesi eklemeniz gerekir.
- Kalıtımı (inheritance) gereksiz yere kullanmayın.
  - Ortak alan az ise (ör. yalnızca ad ve soyad) ayrı sınıflarda tanımlamak yeterlidir. Association çoğu durumda daha doğrudur.
- Referansın `null` olup olmadığını kontrol edin.
  - Kontrol yapılmazsa `NullPointerException` alınır ve program kapanır.
- Sağlam kod (solid code) yazmak programcının sorumluluğudur.

Kısa Tekrar Notları

- **Kopya yapıcı:** Parametre olarak kendi sınıfından bir nesne alan yapıcı. Ör. `Kitap(Kitap k)`.
- **`this(...)`:** Aynı sınıfın başka bir yapıcısını çağırır. Derste ilk satırda yazıldı; sürüm farkı için ders sonrası notlara bakın.
- **Sığ kopya:** İlkel alanların değeri, referans alanların yalnızca adresi kopyalanır.

- **Association:** "has-a" (sahiplik) ilişkisi. Karşı sınıfın referansı bir **üye alan** (attribute) olarak tutulur.
- **Dependency:** Karşı sınıfın nesnesi yalnızca bir **metodun içinde** (parametre veya geçici değişken olarak) kullanılır. Sahiplik yoktur.
- **Ok yönü:** Kimin kime mesaj (metot çağrısı) gönderebileceğini gösterir. Ok yoksa ilişki çift yönlü olabilir veya yön düşünülmemiştir.
- **Kardinalite:** `0..1`, `1`, `0..*` (*), `1..*`, `1..11` gibi gösterilir.
- **One-way association:** `Person` içinde `Car` referansı var, `Car` içinde `Person` yok.
- **Two-way association:** İki sınıf da birbirinin referansını tutar.
- **Yapıcı ve sonlandırıcı:** Diğer metotların çalışma sırası belli değildir. Yapıcı nesne oluşurken, sonlandırıcı (finalizer) nesne silinirken kendiliğinden çalışır.

Detaylı Açıklamalar

Proje export ve import

- Export:
  - NetBeans: `File > Export Project > To ZIP`. Hedef dosya adına `.zip` uzantısını eklemeyi unutmayın. Hoca derste uzantıyı yazmadığı için ilk denemede hata aldı.
  - Import: `File > Import Project > From ZIP`.
  - Projeyi taşırken karakter kodlamasının UTF-8 olmasına dikkat edin.
- Projenizin diskteki yeri: projeye sağ tık `> Properties`. NetBeans'te projeler genellikle `Belgeler/NetBeansProjects` altındadır.

Kopya yapıcı (copy constructor)

Derste `Kitap` sınıfı şu hâldeydi (kareden):

```
public class Kitap {
    private int ID;
    private String ad;
    private Yazar yazar;
    static int id = 0;

    public Kitap(int ID, String ad) {
        this.ID = ID;
        this.ad = ad;
    }

    public Kitap(int ID, String ad, Yazar yazar) {
        this(ID, ad);
        this.yazar = yazar;
    }

    // Kopya yapıcı
    public Kitap(Kitap k) {
        this(k.ID, k.ad);
        this.yazar = k.yazar;
    }

    public void KitapTanit() {
        System.out.println("[ID:" + this.ID + "--->AD:" + this.ad);
        if (this.yazar != null) {
            this.yazar.yazarTanit();
        } else {
            System.out.println("--->Yazar bulunamadı");
        }
    }
}
```

- `this(k.ID, k.ad)` iki parametreli yapıcıyı çağırır. Bu, yapıcı aşırı yüklemesidir (constructor overloading).
- `k.ID` private olsa da aynı sınıfın içinden erişilebilir.
- Eski nesnede yazar atanmamışsa `k.yazar` zaten `null`'dır. Yeni nesneye de `null` geçer.
- Kullanım: `Kitap k5 = new Kitap(k);`

Derste yapılan deneme:

- `k5` oluşturulduktan sonra `k5.setAd("Java 5")`, `k5.setID(5)`, `k5.setYazar(y2)` çağrıldı.
- Orijinal `k` değişmedi. Çünkü `ID` ve `ad` alanlarının kendisi değil, kopyası değişti.
- Bir öğrenci sordu: `k2`'nin yazarı `k5`'e kopyalandıktan sonra `k2` üzerinden yazar değiştirilirse `k5` etkilenir mi?
  - `k2.setYazar(y3)` ile **referans** değiştirilirse `k5` etkilenmez. `k5` hâlâ eski yazarı gösterir.
  - `k2.getYazar().setAd("küçük Ali")` ile **yazarın kendisi** değiştirilirse iki kitapta da değişiklik görülür. Çünkü iki kitap aynı `Yazar` nesnesini gösterir.

C++'ta kopya yapıcı

- C++'ta sınıf tanımında `public:` ve `private:` blokları kullanılır. Java'daki `package` yerine `namespace` vardır.
- Kopya yapıcı, nesneyi referansla (`&`) alır: `Line(const Line &obj);`
- Yıkıcı `~` ile tanımlanır. Bellek `delete` ile programcı tarafından boşaltılır.
- `Line line2 = line1;` satırı C++'ta kopya yapıcıyı çalıştırır.
- Java'da pointer yoktur. C#'ta `unsafe` bölgede pointer kullanılabilir.

Üniversite bilgi sistemi senaryosu

Öğrencilerle birlikte senaryodan sınıflar ve özellikler çıkarıldı:

- **Üniversite:** Fakülteler dizisi, rektör (öğretim üyesi referansı).
- **Fakülte:** Bölümler dizisi, dekan (öğretim üyesi referansı), derslikler (bloklar).
- **Öğretim üyesi:** ID, ad, soyad, bölüm, yetkiler.
  - Rektör ve dekan için ayrı sınıf açılmaz. Bunlar birer öğretim üyesidir.
  - Rektör, dekan ve bölüm başkanı gibi roller için ayrı bir **Yetki** sınıfı (yetki ID'si, yetki adı) tanımlanabilir.
  - Rollere ayrı metotlar gerekiyorsa kalıtım düşünülebilir.
- **Derslik:** Kapı numarası veya ID, ad, tür (laboratuvar, konferans salonu), kapasite, malzemeler dizisi (projeksiyon, tahta).
- **Öğrenci:** Öğrenci numarası, ad, soyad, bölüm referansı, danışman (öğretim üyesi referansı), aldığı dersler (açılan ders dizisi).
- **Katalog ders:** Bölüm kataloğundaki ders (kod, ad, kredi, bölüm).
- **Açılan ders:** O dönem açılan şube. Kendi ID'si, grup kodu, öğretim elemanı.
  - Açılan ders katalog dersten kalıtım almaz. Aralarında association kurulur.
- Bölüm sınıfında öğrenci listesi tutmak şart değildir. İki yönlü ilişki ek iş yükü getirir.
- Hocanın kalıtıma karşı uyarısı:
  - Kalıtımda üst sınıfın yapıcısı da çalışır. Bu ek maliyettir.
  - Yetki gibi konular kalıtımla çözülürse güvenlik açığı oluşabilir.

Adres defteri örneği

- Kaynak: Gordon College "Address Book Example" sayfası (`cs.gordon.edu/courses/cs211/AddressBook`
- Sayfada bir projenin aşamaları sırayla var: gereksinimler (requirements), use case, sequence diagram, class diagram, detaylı sınıf tasarımı ve kod.
- Mesaj: Kod yazmadan önce sınıflar, özellikler ve ilişkiler belirlenmelidir.

Nesneler arası ilişkiler

- Beş ilişki türü: association, dependency, aggregation, composition, inheritance.
- **Association:**
  - Örnek: Bir müşteri (`Customer`) birden çok sipariş (`Order`) verebilir.

  - `Customer` sınıfında `private` bir `Order` dizisi tutulur.
  - "Siparişlerimi listele" gibi bir metot, her siparişin `introduceOrder()` metodunu çağırır. Bu çağrıya **mesaj** denir.
  - İlişkinin özü sahipliktir (has-a): `Customer` has `Orders`.
- **Dependency:**
  - A, B'yi yalnızca bir metodunun içinde kullanır (parametre veya geçici değişken). Gösterim kesikli oktur.
  - Bazı kitaplarda association ile dependency neredeyse aynı kabul edilir.
- **Ok yönü:** Çift yönlüde iki sınıfa da karşı tarafın referansı (veya dizisi) eklenir.
- **Kardinalite:** `0..*`, `1..*`, `1`, `1..11` gibi yazılır. Sabit üst sınır yerine genelde * kullanılır.
- Veritabanındaki crow's foot gösterimi ile UML kardinalitesi benzerdir. Çoka çok ilişkide veritabanında üçüncü tablo gerekir. Java'da bunun için ek sınıf gerekmez.

One-way association: `Person` ve `Car`

Slayttaki kod (kareden):

```
package nyp02;
public class Person {
    private String name;
    private Car car;

    public Person(String name) {
        this.name = name;
    }
    public String getName() { return name; }
    public Car getCar() { return car; }
    public void setCar(Car car) { this.car = car; }

    public String introduceSelf() {
        String intro;
        intro = "Hello, my name is " + getName();
        if (car != null)
            intro += " and I have a car with license plate "
                  + car.getPlate() + ".";
        return intro;
    }
}
```

- `Person` ile `Car` arasındaki kardinalite `0..1`. Her kişinin arabası olmak zorunda değil.
- Gizli bilgi: `Car(String)` yapıcısına gelen metin `plate` alanına atanmalıdır. Plaka araba oluşturulurken verilir ve değişmez.
- `car` yapıcıda atanmazsa `null` kalır. Bu yüzden `introduceSelf()` içinde `if (car != null)` kontrolü şarttır.
  - Kontrol kaldırılıp arabası olmayan bir kişi için çağrılırsa `NullPointerException` alınır. Hoca bunu derste çalıştırarak gösterdi.
- `aziz.getCar().getPlate()` yazmak derleme açısından yasaldır. Ama araba atanmamışsa çalışma zamanında `NullPointerException` verir.
- Sequence diagram: `main`, `Person` nesnesine `introduceSelf` isteği gönderir. `Person` önce kendi `getName`'ini, sonra `Car` nesnesinin `getPlate`'ini çağırır. Araba yoksa yalnızca ad döner.
- Diyagramda boş elmaslı (aggregation) gösterim, A'nın içinde B nesnelerinden oluşan bir dizi olduğu anlamına gelir.

Two-way association

- Kişi arabayı, araba da sahibini (`owner`) tutar. İki sınıfa da `get/set` metotları eklenir (`getOwner`, `setOwner`).
- Örnek kullanım: Elinizde yalnızca araba nesnesi varsa, sahibine (ör. kiralama firması) araba üzerinden ulaşabilirsiniz.
- Ayrıntılar bir sonraki derse kaldı.

Ders sonrası notlar

- Hoca "copy constructor Java'da olan bir kavram değil, C++'ta var" dedi. Daha doğru ifade şu: Java'da derleyicinin otomatik ürettiği bir kopya yapıcı yoktur. Kopya yapıcı, derste yapıldığı gibi elle yazılan bir yapıcıdır. C++'ta ise derleyici yoksa kendisi üretir.
- `.class` dosyası: Hoca "bunları biz açamayız" dedikten sonra decompiler ile açılabildiğini, ama tam geri dönüştürülemediğini söyledi (00:15:27-00:15:37).
- `this(...)` ilk satır kuralı: Altyazıda bu kural geçmiyor. JDK 25'te `this(...)` öncesinde başka deyim yazılabilir: Java ile denendi, derlendi. `-source 21` ile aynı kod hata veriyor.
- C++ kopya ve atama: `Line line2 = line1;` ilk değer vermedir, kopya yapıcıyı çalıştırır. Var olan nesneye yazılan `line3 = line1;` ise `operator=` işlecini çalıştırır (g++ ile denendi).
- `finalize`: JDK 25'te kullanımdan kaldırılmış ve silinmek üzere işaretlidir (`forRemoval=true`); Java ile denendi. Sonlandırıcının çalışmasına güvenilmez.
- Hoca, kopyalamada string alanlar için "string havuzundan yeni bir string oluşturuluyor ve kopyalanıyor" dedi. Doğrusu: `this.ad = k.ad` yalnızca **referansı** kopyalar. İki nesne aynı `String` nesnesini gösterir. Bu derin kopya değil, sığ kopyadır. `String` değiştirilemez (immutable) olduğu için bu bir sorun yaratmaz. `k5.setAd(...)` yeni bir string referansı atar, orijinali etkilemez.
- Hoca, `ID` gibi ilkel alanlar için "direkt deep copy yapılıyor" dedi. İlkel tiplerde kopyalanan şey değerin kendisidir. Deep copy/shallow copy ayrımı referans tipli alanlar için anlamlıdır. Bu kopya yapıcı sığ kopya (shallow copy) yapar. Derin kopya için `this.yazar = new Yazar(k.yazar)` gibi yazarın da kopyası oluşturulmalıdır.
- Hoca, `Integer i;` tanımlayıp `i = i + 5` yazmanın "null pointer exception veya initialization exception" vereceğini söyledi. İki ayrı durum vardır (Java ile denendi):
  - **Yerel değişken** (`Integer i;` veya `int i;` metot içinde) ilk değer almaz. `i = i + 5` **derleme hatası** verir: `variable i might not have been initialized`.
  - **Sınıf alanı** olan `Integer i` varsayılan olarak `null` olur. `i = i + 5` çalışma zamanında `NullPointerException` fırlatır. `int` tipindeki alan ise `0` olur.
- Hoca kalıtımda yapıcının "iki defa çağrıldığını" söyledi. Kastedilen, alt sınıf nesnesi oluşturulurken önce üst sınıfın, sonra alt sınıfın yapıcısının çalışmasıdır. Aynı yapıcı iki kez çalışmaz.
- Altyazıdaki "Avesiz/Abisis" Avesis, "NetBeast" NetBeans, "InterC" IntelliJ, "Bulobna/Bolugna" Bologna, "palaces" Avesis sayfası demektir.
- 02:33 civarında altyazıda "Aziz Sancar'a bir soru… İntroduğum" gibi tekrar eden satırlar var. Bu kısım sequence diagram'ın okunmasıdır; altyazı bozuktur.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.